



JÖNKÖPING UNIVERSITY
School of Engineering

Lab report

Lab package 1

Firstname Lastname, Firstname Lastname, Firstname Lastname



v. 1.0

Summary: Far too often students are tormented by Word or other word processors that cannot handle equations, figures or tables in a reliable and satisfying way. This template shows how Quarto and Jupyter Notebooks can produce good-looking PDF reports through LaTeX. The content ends up in focus, not the formatting. Study this document and its cells to learn how it is put together. Keep the report tidy, short and to the point!

1 Introduction: how to write modern reports

This template uses [Quarto](#) together with Jupyter Notebooks to produce PDF reports through LaTeX. Unlike Overleaf, we write in ordinary Jupyter Notebooks and use LaTeX *only* for mathematical equations. Quarto takes care of all the formatting, the numbering of sections, equations and figures, and the references, automatically.

1.1 Installation

To render this template you need:

1. **Python and Jupyter**, see the [installation guide](#)
2. **Quarto** from [quarto.org](#). Choose “Install for me only” if you do not have administrator rights.
3. **A LaTeX distribution** for PDF rendering. Check whether you already have one with `pdflatex --version`. If not:
 - Linux: `sudo apt install texlive-full`
 - macOS: [MacTeX](#)
 - Windows: [MiKTeX](#)
 - Or TinyTeX (small, about 250 MB): `quarto install tinytex`
4. **VS Code** with the [Quarto](#) and [Jupyter](#) extensions

See [PDF Reports with LaTeX](#) for a full walkthrough of Quarto, LaTeX maths and troubleshooting.

1.2 Workflow

1. Open this notebook in VS Code
2. Edit the text in markdown cells, add equations with LaTeX syntax
3. Add Python code cells for figures and calculations
4. Render to PDF with one of the following commands:

```
quarto preview report.ipynb --to pdf
```

`quarto preview` renders the PDF and opens it automatically. It also watches the file, so every time you save the notebook the PDF is re-rendered and updated. Perfect while writing, just like Overleaf’s live preview. Stop it with `Ctrl+C` in the terminal.

```
quarto render report.ipynb --to pdf --execute
```

`quarto render` renders the PDF once, without watching. The `--execute` flag runs every code cell during rendering, which makes the results reproducible. Use this for the final version.

1.3 Document structure

The whole report lives in a single notebook file. The metadata (title, authors, format) is defined in the YAML block in the first cell (a *raw* cell). Markdown cells hold text and equations, and code cells generate figures and tables automatically.

2 Figures

See Figure 1, a graph of the very highest quality.



Figure 1: Look at this graph

2.1 How do we handle figures?

Look at the source of the notebook cells to see how a figure is included with a caption and a reference. The syntax is:

```
![Caption](path/to/image.jpg){#fig-label width=60%}
```

Then refer to it in the text with `@fig-label`. Quarto numbers the figures for you.

We always include a caption and describe the figure properly. If the figure is not our own, we cite the source.

2.2 Vector graphics

Always aim for vector images (SVG, PDF), which can be scaled freely without losing quality. You can use PowerPoint or [Inkscape](#) to draw graphics and export them as SVG.

2.3 Figures from Python code

One of Quarto's great strengths is that Python code can generate figures directly in the report. The code cell below generates Figure 2 using these Quarto comments:

```
#| label: fig-sinewave
#| fig-cap: "A sine wave generated with Python"
#| echo: true
```

- `label: fig-sinewave` makes the figure referable with `@fig-sinewave`
- `fig-cap` sets the caption
- `echo: true` shows the code in the PDF (set it to `false` to hide it)

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(0, 2*np.pi, 100)
plt.plot(x, np.sin(x))
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.grid(True, alpha=0.3)
plt.show()
```

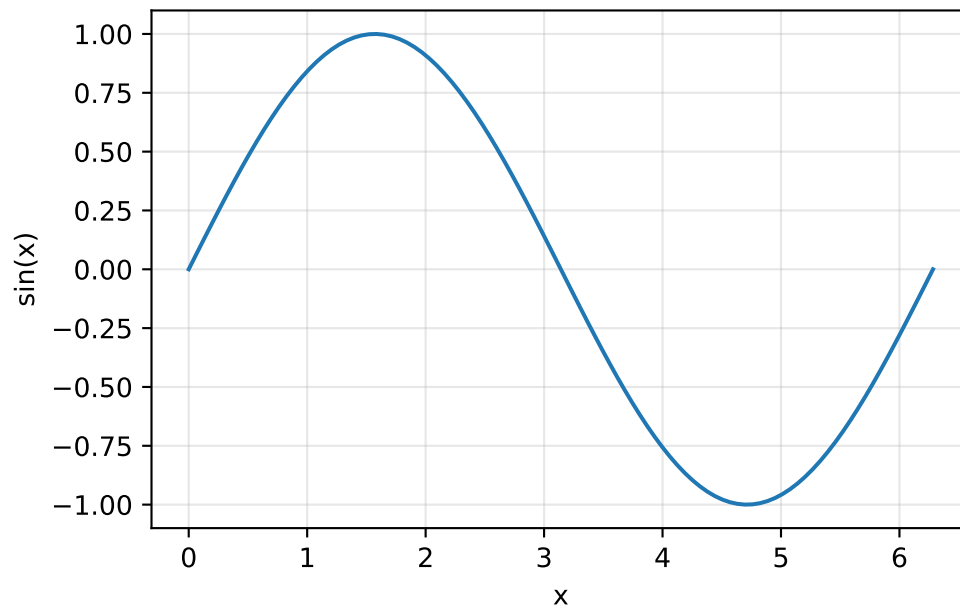


Figure 2: A sine wave generated with Python

The code cell above uses Quarto comments (`#|`) to control how it appears, see Table 1.

Table 1: Quarto comments for code cells

Option	Effect
<code># echo: true</code>	Show the code (default)
<code># echo: false</code>	Hide the code, show only the figure
<code># output: false</code>	Hide the output
<code># warning: false</code>	Suppress warnings
<code># label: fig-name</code>	Make the figure referable
<code># fig-cap: "..."</code>	Caption

3 Mathematical notation

“An equation says more than a thousand figures”

— Fredrik Larsson

An engineer has to be precise in their language to communicate efficiently and avoid misunderstandings. The point of being precise is to let the reader follow the reasoning and reproduce the work. Below is the kind of mathematical notation a mechanical engineer inflicts on their surroundings.

3.1 Equations in text and on their own line

Use single dollar signs $\$ \dots \$$ for maths in running text and double ones $\$\$ \dots \$\$$ for centred equations.

The model rests on kinetic energy being converted into potential energy plus some kinetic energy: the train has an initial velocity v_0 that takes it up to the height h , where it has the velocity v .

$$\frac{1}{2}mv_0^2 = mgh + \frac{1}{2}mv^2$$

Solving for v

$$v = \sqrt{v_0^2 - 2gh} \tag{1}$$

In the positive normal direction we get

$$N - mg \cos \theta = ma_c$$

The normal acceleration along a curved path is given by

$$a_c = \frac{v^2}{r}$$

Substitution gives

$$N = \frac{mv^2}{r} + mg \cos \theta$$

and we divide out the mass and g to express it as a number of g “forces”

$$\frac{N}{mg} = \frac{v^2}{rg} + \cos \theta =: G$$

A constant normal acceleration implies

$$a_c = Cg = \frac{v^2}{r} \quad (2)$$

Putting Equation 1 into Equation 2

$$a_c = Cg = \frac{v_0^2 - 2gy}{Cg}$$

$$\stackrel{\text{let } h=y}{\Rightarrow} \boxed{r = \frac{v_0^2 - 2gh}{Cg}}$$

The relation between arc length, radius and angle is

$$ds = r d\theta \Rightarrow \frac{d\theta}{ds} = \frac{1}{r}$$

which leads to a final, rather pleasant ordinary differential equation, solved numerically with Python.

$$\text{ODE} \begin{cases} \frac{d\theta}{ds} = \frac{1}{r} \\ \frac{dx}{ds} = \cos \theta \\ \frac{dy}{ds} = \sin \theta \end{cases}$$

3.2 Vectors and matrices

- Vectors are normally set in bold: $\mathbf{a}$ and $\boldsymbol{\varphi}$, or with `\bm`: $\boldsymbol{\sigma}$.
- Italics for tensors: $\mathcal{T}, \mathcal{C}_{ijkl}$. A matrix can be written $\mathbf{A}$ and its numerical version $\mathbf{A}$.
- There are plenty of ways to write a transpose. The lazy one: $\mathbf{a}^T$, the less lazy $\mathbf{a}^\top$, and the arguably prettier $\mathbf{a}^\top$.
- Force vectors are written $\mathbf{F}$ or $\mathbf{F}$

The basis function matrix is defined as

$$\boldsymbol{\Phi} := \begin{bmatrix} \varphi_1 & 0 & 0 & \varphi_2 & 0 & 0 & \cdots \\ 0 & \varphi_1 & 0 & 0 & \varphi_2 & 0 & \cdots \\ 0 & 0 & \varphi_1 & 0 & 0 & \varphi_2 & \cdots \end{bmatrix} \quad (3)$$

3.3 Large matrices

Matrices containing fractions sometimes need more room between the rows. Here we use raw LaTeX:

$$\mathbf{J}(\xi, \eta, \zeta) = \begin{bmatrix} \frac{\partial x_h}{\partial \xi} & \frac{\partial y_h}{\partial \xi} & \frac{\partial z_h}{\partial \xi} \\ \frac{\partial x_h}{\partial \eta} & \frac{\partial y_h}{\partial \eta} & \frac{\partial z_h}{\partial \eta} \\ \frac{\partial x_h}{\partial \zeta} & \frac{\partial y_h}{\partial \zeta} & \frac{\partial z_h}{\partial \zeta} \end{bmatrix}$$

3.4 Numerical evaluation

Sometimes a calculation or the evaluation of a function has to be written out:

$$\frac{dy}{dt} = \dot{y} = \frac{125\pi}{200} \sin\left(\frac{50\pi}{200}\right) \cdot 150 = \frac{375\pi\sqrt{2}}{8} \approx 208 \text{ m/s}$$

Every time the symbol $*$ is misused as a multiplication sign, a mathematician or an engineer weeps a little on the inside.

$$f * g$$

means the convolution¹ of f with g .

3.5 Gaussian integration

$$\int_0^1 f(x) dx \approx \sum_{i=1}^n f(x_i)w_i$$

An n -point Gaussian quadrature is exact for polynomials of degree $2n - 1$ or lower. See Table 2.

Table 2: Gaussian quadrature for $\xi \in [0, 1]$

Gauss order n	Polynomial degree	Weights w_i	Points x_i
1	1	1	$\frac{1}{2}$
2	3	$\frac{1}{2}, \frac{1}{2}$	$\frac{1}{2} \pm \frac{\sqrt{3}}{6}$
3	5	$\frac{5}{18}, \frac{4}{9}, \frac{5}{18}$	See below

Gauss points for $n = 3$: $\frac{1}{2} - \frac{\sqrt{15}}{10}, \frac{1}{2}, \frac{1}{2} + \frac{\sqrt{15}}{10}$.

For arbitrary Gauss orders and intervals you can use the Golub-Welsch algorithm.

4 Tables

4.1 Simple tables in markdown

Column 1	Column 2	Column 3
Data	Data	Data
Data	Data	Data

4.2 Complex tables with LaTeX

For tables containing equations or special formatting you can use raw LaTeX. With `\resizebox{\textwidth}{!}{...}` the table is scaled to the width of the text:

State of stress	Boundary Conditions	von Mises Equations
General 3D	No restrictions	$\sigma_v = \sqrt{\frac{1}{2} \left[(\sigma_x - \sigma_y)^2 + (\sigma_y - \sigma_z)^2 + (\sigma_z - \sigma_x)^2 + 6(\sigma_{xy}^2 + \sigma_{yz}^2 + \sigma_{zx}^2) \right]}$
General plane stress (2D)	$\sigma_z = \sigma_{yz} = \sigma_{zx} = 0$	$\sigma_v = \sqrt{\sigma_x^2 - \sigma_x \sigma_y + \sigma_y^2 + 3\sigma_{xy}^2}$

¹<https://en.wikipedia.org/wiki/Convolution>

5 Including code in reports

In Quarto, Python code is shown straight from the code cells. Control what is displayed with `#|` comments at the top of the cell:

- `#| echo: true` shows the code (default)
- `#| echo: false` hides the code and shows only the result
- `#| output: false` hides the result

Below is a code example with `#| echo: true`:

```
import numpy as np

# Element loop example
n_elements = 4
n_nodes = n_elements + 1
S = np.zeros((n_nodes, n_nodes))
F = np.zeros(n_nodes)

for iel in range(n_elements):
    inod = [iel, iel + 1]      # Element nodes
    Sk = np.array([[1, -1],
                   [-1, 1]])  # Element stiffness
    fk = np.array([0.5, 0.5]) # Element force

    # Assembly
    ix = np.ix_(inod, inod)
    S[ix] += Sk
    F[inod] += fk

print("Stiffness matrix:")
print(S)
print("Force vector:", F)
```

```
Stiffness matrix:
[[ 1. -1.  0.  0.  0.]
 [-1.  2. -1.  0.  0.]
 [ 0. -1.  2. -1.  0.]
 [ 0.  0. -1.  2. -1.]
 [ 0.  0.  0. -1.  1.]]
Force vector: [0.5 1.  1.  1.  0.5]
```

6 References

Quarto handles references through `.bib` files. In the YAML block we write:

```
bibliography: library.bib
```

Then cite with `@key` or `[@key]`:

- `@Stromberg2012` gives: N. Strömberg (2012)
- `[@Johannesson2013; @Logan2012]` gives: (H. Johannesson, J-G Persson and D Pettersson 2013; D. L. Logan 2012)

Sometimes you need to refer to the same clever people who did good things, see (H. Johannesson,

J-G Persson and D Pettersson 2013; D. L. Logan 2012; B. Lundkvist 1997; N. Strömberg 2012).

6.1 Reference style

The reference style is changed with a CSL file in the YAML. For IEEE style, for example:

```
csl: ieee.csl
```

Download `ieee.csl` from the [Zotero Style Repository](#) and put the file in the same folder as the notebook. Common styles:

Style	File name	Format
IEEE	<code>ieee.csl</code>	[1], [2], [3]
APA	<code>apa.csl</code>	(Strömberg, 2012)
Vancouver	<code>vancouver.csl</code>	(1), (2), (3)

6.2 BibTeX format

The reference data is stored in `library.bib` in BibTeX format:

```
@Book{Callister2011,  
  author = {W. D Callister and D. G Rethwisch},  
  title  = {Materials Science and Engineering},  
  year   = {2011},  
  volume = {SI version},  
  edition = {8},  
}
```

You can search for “title + author bibtex” or use the cite button in Google Scholar and pick the BibTeX link.

Every cited reference is added automatically at the end of the document.

References

B. Lundkvist. 1997. *Ritsteknik*. 7th ed. Liber.

D. L. Logan. 2012. *A First Course in the Finite Element Method*. Fifth. Global Engineering.

H. Johannesson, J-G Persson and D Pettersson. 2013. *Produktutveckling: Effektiva Metoder För Konstruktion Och Design*.

N. Strömberg. 2012. *Nonlinear FEA and Design Optimization for Mechanical Engineers*. 3rd ed.